SentinelVision AI

Volume 01

Product Requirements Document

Suite Version	1.0.0
Classification	Internal / Confidential
Generated	2026-07-17 07:23 UTC

volume: 01

title: Product Requirements Document

status: draft

> **SentinelVision AI — Enterprise Documentation Suite**

> Status legend: Implemented | Planned | Future

1. Document Control

Field	Value
Document ID	DOC-PRD-001
Version	1.0.0
Status	Approved for phased delivery

2. Executive Summary

SentinelVision AI transforms CCTV infrastructure into an intelligent security and activity monitoring platform using hybrid deployment: AI inference at the customer edge, centralized control plane for operations, reporting, and governance.

2.1 Current MVP (M0/M1) — Implemented

- Multi-tenant Django REST API with JWT authentication
- Organization, site, zone, camera, edge device registry
- Simulated edge person-detection event ingestion
- Dashboard summary, camera inventory, event review, alert acknowledgement
- Docker Compose local stack and basic CI

2.2 Target Platform — Planned/Future

- Production YOLO, tracking, InsightFace, threat detection modules
- Visitor management, attendance, reports, notifications
- Object storage, Celery jobs, Kubernetes production deployment
- Enterprise SSO, audit automation, biometric consent workflows

3. Business Goals

ID	Goal	KPI	Target
BR-01	Reduce incident response time	Mean time to acknowledge critical alert	< 2 min
BR-02	Improve threat detection quality	Intrusion/weapon true positive rate	>= 92%
BR-03	Automate attendance	Manual reconciliation hours/week	-80%
BR-04	Visitor compliance	Pre-registered check-in accuracy	>= 98%

ID	Goal	KPI	Target
BR-05	Platform reliability	Control plane uptime	99.9%
BR-06	Tenant growth	Active sites per customer	>= 3
BR-07	Privacy compliance	Biometric consent coverage	100%

4. Product Vision

Every camera becomes an intelligent sensor protecting people, assets, and operations with privacy-respecting AI that security teams can trust.

5. Product Scope

5.1 In Scope

Area	Status
Multi-tenant org/site/zone management	Implemented
Camera and edge device registry	Implemented
Event ingestion and review queue	Implemented
Alert rules and acknowledgement	Partial — rules create alerts; notifications Planned
Web dashboard operations UI	Implemented (core screens)
Simulated edge inference	Implemented
Face recognition and attendance	Planned
Weapon/fire/smoke detection	Future
Native mobile apps	Future

5.2 Out of Scope (MVP)

- Full VMS/NVR replacement
- Continuous cloud video archive
- Automated law enforcement dispatch
- Custom model training portal (Future)

6. User Personas

Persona	Role	Primary Goals
Sarah	Security Manager	Threat response, incident audit
Ahmed	Facility Admin	Camera config, uptime, user admin
Priya	HR Manager	Attendance, visitor compliance
James	Edge Technician	Edge install, diagnostics

Persona	Role	Primary Goals
Maria	Executive	Risk dashboards, compliance

7. User Stories

ID	Story	Priority	Status
US-01	Real-time intrusion alerts	P0	Planned
US-02	Review events with evidence	P0	Partial
US-03	Unknown person in restricted zone	P0	Planned
US-04	Weapon/fire/smoke severity alerts	P0	Future
US-05	Register cameras to sites/zones	P0	Implemented
US-06	RBAC for configuration changes	P0	Implemented
US-07	Edge device health monitoring	P1	Implemented
US-08	Automated attendance	P1	Planned
US-09	Visitor pre-registration	P1	Planned
US-10	Biometric consent before enrollment	P0	Planned
US-11	Weekly incident reports	P1	Planned
US-12	Immutable audit logs	P0	Partial — model exists, auto logging Planned

8. Functional Requirements

8.1 Camera Management (FR-CAM)

ID	Requirement	Status
FR-CAM-01	Register camera with RTSP URL, site, zone	Implemented
FR-CAM-02	ONVIF discovery	Planned
FR-CAM-03	Camera online/offline status from edge heartbeat	Implemented
FR-CAM-04	Assign AI modules per camera	Partial — field exists
FR-CAM-05	Configure detection zone polygons	Implemented (data model)

8.2 Dashboard (FR-DASH)

ID	Requirement	Status
FR-DASH-01	Overview metrics	Implemented
FR-DASH-02	Event review queue with filters	Partial
FR-DASH-03	Live camera status grid	Planned
FR-DASH-04	Role-based navigation	Partial

8.3 Alerts & Notifications (FR-ALERT)

ID	Requirement	Status
FR-ALERT-01	Configurable alert rules	Implemented
FR-ALERT-02	Severity levels info/warning/critical	Implemented
FR-ALERT-03	Email/SMS/push notifications	Planned
FR-ALERT-04	Acknowledgement workflow	Implemented

8.4 Reports (FR-RPT)

ID	Requirement	Status
FR-RPT-01	Incident summary export PDF/CSV	Planned
FR-RPT-02	Attendance report	Planned
FR-RPT-03	Visitor log report	Planned
FR-RPT-04	Audit log export	Planned

8.5 Mobile (FR-MOB)

ID	Requirement	Status
FR-MOB-01	Responsive web for alerts	Implemented
FR-MOB-02	PWA push notifications	Planned
FR-MOB-03	Native iOS/Android apps	Future

9. Non-Functional Requirements

ID	Category	Requirement	Target	Status
NFR-01	Performance	Event ingest latency p95	< 3s	Planned measurement
NFR-02	Performance	Dashboard load	< 2s	Planned measurement
NFR-03	Scalability	Cameras per edge node	16	Target
NFR-04	Availability	Control plane uptime	99.9%	Future

ID	Category	Requirement	Target	Status
NFR-05	Security	TLS in transit	1.2+	Planned production
NFR-06	Security	AES at rest	Required	Planned
NFR-07	Privacy	Biometric consent mandatory	100%	Planned
NFR-08	Retention	Event metadata default	90 days	Planned enforcement
NFR-09	Accessibility	WCAG 2.1 AA	Target	Future

10. AI Modules Overview

Module	Status	Phase
Multi-person tracking	Planned	M2
Face recognition	Planned	M3
Unknown person detection	Planned	M3
Intrusion detection	Planned	M2
Activity recognition	Future	M4
Fire/smoke detection	Future	M4
Weapon detection	Future	M4
Attendance	Planned	M3
Visitor management	Planned	M3
Vehicle detection	Future	M5
License plate recognition	Future	M5

11. Acceptance Criteria

11.1 M0/M1 Foundation — Implemented

- JWT login and tenant-scoped APIs operational
- Camera/site CRUD with RBAC enforced on writes
- Edge heartbeat and event ingest functional
- Dashboard displays summary, cameras, events, alerts
- Docker Compose stack starts successfully
- CI runs backend, edge, frontend, and documentation builds

11.2 M2 Core AI — Planned

- One RTSP stream connected at edge
- YOLO person detection with tracking IDs
- Zone-based intrusion events generated
- Operator confirms event and acknowledges alert end-to-end

12. Known Gaps (Remediation Required)

Gap	Impact	Remediation
Audit auto-logging not wired	Compliance gap	Implement log_audit callers
Global edge_event_id lookup	Tenant isolation risk	Scope idempotency per organization
Cross-site zone assignment possible	Data integrity risk	Add serializer validation
Fallback EDGE_API_KEY without device binding	Security risk	Remove or bind to bootstrap device
Notification dispatch absent	Operational gap	Add Celery notification service

13. Roadmap

Phase	Timeline	Deliverables
M0/M1	Complete	Foundation, simulated pipeline
M2	Weeks 4-10	RTSP, YOLO, intrusion, alerts
M3	Weeks 11-14	Face, attendance, visitors
M4	Weeks 15-18	Threat modules, reports
M5	Weeks 19-24	Enterprise HA, SSO, analytics

14. Traceability

See `docs/traceability-matrix.csv` for BR/FR/NFR to UC/API/DB/TEST mappings.

SentinelVision AI

Volume 02

Software Requirements Specification

Suite Version	1.0.0
Classification	Internal / Confidential
Generated	2026-07-17 07:23 UTC

volume: 02

title: Software Requirements Specification

status: draft

> **SentinelVision AI — Enterprise Documentation Suite**

> Status legend: Implemented | Planned | Future

1. System Overview

SentinelVision AI comprises an edge inference worker, Django REST control plane, React dashboard, PostgreSQL datastore, and planned async/observability services.

1.1 Actors

Actor	Description
SecurityOperator	Monitors alerts and reviews events
FacilityAdmin	Configures cameras, sites, users
HROfficer	Manages attendance and visitors
EdgeService	Automated edge worker posting heartbeats/events
SystemAuditor	Reviews audit and compliance exports
PlatformAdmin	Operates control plane infrastructure

2. Use Cases

ID	Use Case	Primary Actor	Status
UC-AUTH-01	Authenticate with JWT	All users	Implemented
UC-CAM-01	Register camera	FacilityAdmin	Implemented
UC-EDGE-01	Edge heartbeat registration	EdgeService	Implemented
UC-EVT-01	Ingest detection event	EdgeService	Implemented
UC-EVT-02	Review detection event	SecurityOperator	Implemented
UC-ALERT-01	Acknowledge alert	SecurityOperator	Implemented
UC-ALERT-02	Configure alert rule	FacilityAdmin	Implemented
UC-ATT-01	Record attendance via face	EdgeService	Planned
UC-VIS-01	Pre-register visitor	HROfficer	Planned
UC-RPT-01	Export incident report	SystemAuditor	Planned

UC-EVT-01 Ingest Detection Event

Preconditions: Edge device authenticated; camera active.

Main flow:

1. Edge samples inference result.

2. Edge POSTs event payload to `/api/v1/events/ingest/`.
3. API validates camera ownership/site binding.
4. API persists DetectionEvent.
5. API evaluates alert rules and creates Alert records when matched.

Alternate flows:

- A1: Duplicate `edge_event_id` returns existing event (200).
- A2: Invalid camera returns 404.
- A3: Camera outside edge site returns 403.

Postconditions: Event stored; optional alert created.

UC-EVT-02 Review Detection Event

Preconditions: Operator authenticated with Operator+ role.

Main flow:

1. Operator opens event review queue.
2. Operator selects pending event.
3. Operator marks confirmed or false positive.
4. System records reviewer and timestamp.

3. UML Use Case Diagram

```
flowchart LR
subgraph actors [Actors]
Operator[SecurityOperator]
Admin[FacilityAdmin]
Edge[EdgeService]
HR[HROfficer]
end
subgraph system [SentinelVision]
UC1[ManageCameras]
UC2[IngestEvents]
UC3[ReviewEvents]
UC4[ManageAlerts]
UC5[ManageVisitors]
end
Admin --> UC1
Edge --> UC2
Operator --> UC3
Operator --> UC4
HR --> UC5
```

4. Activity Diagram — Event Processing

1. Edge captures frame
2. Detector produces candidates
3. Module router evaluates zones/rules
4. Event payload assembled
5. HTTP POST with retry
6. API validation
7. Persist event
8. Evaluate alert rules
9. Dashboard refresh

5. Sequence Diagram — Event Ingestion

```
sequenceDiagram
participant Edge
participant API
participant DB
participant Rules as AlertRules
Edge->>API: POST /events/ingest
API->>DB: Validate camera
API->>DB: Insert DetectionEvent
API->>Rules: evaluate_alert_rules
Rules->>DB: Insert Alert optional
API-->>Edge: 201 Created
```

6. State Diagram — Alert Lifecycle

State	Description
open	Created, awaiting operator action
acknowledged	Operator acknowledged
resolved	Incident closed
dismissed	False alarm or non-actionable

Transitions: open→acknowledged (operator), acknowledged→resolved (operator), open→dismissed (operator/admin).

7. Business Rules

ID	Rule
BR-TEN-01	All user queries must be scoped to member organizations
BR-RBAC-01	Viewer cannot mutate configuration
BR-EDGE-01	Edge may ingest only for cameras on its site
BR-ALERT-01	Alert severity must be >= rule min severity
BR-ALERT-02	Event confidence must be >= rule min confidence
BR-BIO-01	Face enrollment requires consent record (Planned)
BR-RET-01	Evidence retention defaults to 30 days (Planned enforcement)

8. Validation Rules

Entity	Field	Validation
Camera	name	Required, unique per site
Camera	rtsp_url	Valid URL format
Zone	polygon_json	Array of normalized [x,y] pairs
DetectionEvent	confidence	0.0–1.0
AlertRule	min_confidence	0.0–1.0
EdgeDevice	api_key	One-time display on create

9. Error Handling

Code	Condition	Response
400	Serializer validation failure	DRF validation errors
401	Missing/invalid JWT or edge key	Authentication failed
403	Role or site mismatch	Permission denied
404	Resource not found	Not found
409	Planned idempotency conflict	Future standardized envelope

Target error envelope (Planned):

```
{"error": {"code": "VALIDATION_ERROR", "message": "...", "details": {}}}
```

10. Workflow Specifications

10.1 Human Review Workflow

Pending → Confirmed | False Positive | Escalated

Escalated events require supervisor review (Planned).

10.2 Edge Offline Workflow

1. Heartbeat missed beyond threshold
2. Edge marked offline (Planned automated rule)
3. Cameras at site marked degraded
4. Operator notification (Planned)

11. Security Requirements

- JWT access token 15 minutes; refresh 7 days
- Edge API keys stored hashed (SHA-256)
- Tenant isolation on all list/detail endpoints
- Planned: rate limiting, key rotation, constant-time compare

12. Traceability

Each UC maps to FR IDs in Volume 1 and API IDs in Volume 5.

SentinelVision AI

Volume 03

System Design Document

Suite Version	1.0.0
Classification	Internal / Confidential
Generated	2026-07-17 07:23 UTC

volume: 03

title: System Design Document

status: engineering-ready

version: 2.0.0

> **SentinelVision AI — Volume 3**

> Status: Implemented | Planned | Future

1. Document Control

Field	Value
Document ID	DOC-SDD-003
Version	2.0.0
Status	Engineering Ready
Architecture style	Modular monolith + edge worker → evolutionary services
Deployment model	Hybrid (edge inference, central control plane)

2. Goals and Non-Goals

2.1 Goals

- Clear blueprint for M2–M6 engineering
- Separate current MVP from target production architecture
- Define RTSP → YOLO → event → alert → storage data path
- Specify Celery/Redis/object storage integration points

2.2 Non-Goals

- Not a full VMS replacement
- Not claiming production AI accuracy in current edge simulation
- Not requiring microservices for M1–M2

3. Current vs Target Architecture

Layer	Current (M0/M1)	Target (M2–M6)
Control plane	Django modular monolith	Same core + extracted workers
Edge	Simulated detector + HTTP	RTSP + OpenCV + YOLO + queue
DB	PostgreSQL / SQLite	PostgreSQL HA
Jobs	Inline alert evaluation	Celery + Redis
Storage	None for media	S3-compatible evidence
Ingress	Compose Nginx	Nginx/Ingress + TLS
Orchestration	Docker Compose	Kubernetes (prod)

Layer	Current (M0/M1)	Target (M2–M6)
Observability	Health endpoint	Prometheus/Grafana/ELK

4. High-Level Architecture

```
flowchart TB
subgraph customerSite [CustomerPremises]
Cams[IPCameras_RTSP]
Edge[EdgeAIService]
Buf[EncryptedLocalQueue]
Cams --> Edge --> Buf
end
subgraph controlPlane [ControlPlane]
Nginx[NginxIngress]
API[DjangoRESTAPI]
Workers[CeleryWorkers]
Redis[(Redis)]
PG[(PostgreSQL)]
S3[(ObjectStorage)]
Web[ReactDashboard]
end
Buf -->|HTTPS_events_heartbeats| Nginx --> API
Web --> Nginx
API --> PG
API --> Redis
Workers --> Redis
Workers --> S3
API --> S3
Workers --> Notify[Email_SMS_Push]
```

4.1 Trust Boundaries

Zone	Trust	Network
Camera LAN	High trust internal	RTSP only to edge
Edge host	Semi-trusted	Outbound HTTPS 443 only
Control plane private	Trusted	DB/Redis not public
Public internet	Untrusted	TLS to Nginx only

5. Django App Architecture (Implemented)

5.1 Project Layout

```
backend/
config/ # settings, urls, wsgi/asgi
apps/
common/ # UUIDModel, TimestampedModel, TenantQuerysetMixin
tenancy/ # Organization, UserMembership, roles
sites/ # Site, Zone
cameras/ # Camera, EdgeDevice, EdgeAPIKeyAuthentication
events/ # DetectionEvent, ingest, review, dashboard, health
alerts/ # AlertRule, Alert, evaluate_alert_rules
audit/ # AuditLog, middleware, log_audit helper
```

5.2 App Responsibilities

App	Responsibility	Key modules
`config`	Django settings, URL mount, JWT/CORS/DRF	`settings.py`, `urls.py`
`common`	Shared base models and tenant mixin	`models.py`, `permissions.py`
`tenancy`	Multi-tenant orgs and RBAC membership	models, views, seed command
`sites`	Facilities and detection zones	Site/Zone CRUD
`cameras`	Camera registry, edge devices, edge auth	authentication, views
`events`	Event ingest, list, review, dashboard	serializers, views
`alerts`	Rules and alert lifecycle	`services.evaluate_alert_rules`
`audit`	Audit storage and list API	middleware context

5.3 Request Processing Pipeline

```
Client
-> SecurityMiddleware
-> CorsMiddleware
-> Session/Common/Csrf
-> AuthenticationMiddleware (Django session)
-> AuditContextMiddleware
-> DRF authentication (JWT or EdgeAPIKey)
-> Permission classes
-> TenantQuerysetMixin scoping
-> Serializer validation
-> ORM / service layer
-> JSON response
```

5.4 Authentication Architecture

```
flowchart LR
User[DashboardUser] -->|POST_auth_token| JWT[SimpleJWT]
JWT -->|Bearer_access| API[DRFViews]
Edge[EdgeWorker] -->|X-Edge-API-Key| EdgeAuth[EdgeAPIKeyAuthentication]
EdgeAuth --> Hash[SHA256Lookup]
Hash --> Device[EdgeDevice]
Device --> API
```

Principal	Auth class	Bound context
Human user	`JWTAuthentication`	`request.user` + memberships
Edge device	`EdgeAPIKeyAuthentication`	`EdgeServiceUser` + optional `device`

5.5 Tenancy Isolation Pattern

```
# Conceptual (implemented in TenantQuerysetMixin)
org_ids = UserMembership.objects.filter(user=request.user).values_list("organization_id")
queryset = Model.objects.filter(organization_id__in=org_ids)
# or via joins: camera__site__organization_id__in=org_ids
```

Write guards: `require_role(user, org, [OWNER, ADMIN, ...])` before create/update/delete.

5.6 Domain Event Path (Current)

```
Edge.detect()
-> POST /api/v1/events/ingest/
-> EventIngestView validates camera + site binding
```

```
-> DetectionEvent.objects.create(...)
-> evaluate_alert_rules(event)
-> optional Alert.objects.create(...)
-> Dashboard/Alerts UI poll lists
```

6. Evolutionary Microservices Plan

Do **not** split for M1–M2. Extract only when boundaries are stable.

Stage	Architecture
M0–M2	Modular monolith + edge
M3	Optional `notification-service` (Celery first is enough)
M4	`report-service` worker; evidence uploader
M5–M6	Optional `inference-coordinator` for multi-site fleets

6.1 Candidate Service Boundaries (Future)

Service	Owns	Talks to
control-api	Tenancy, cameras, events, alerts	PG, Redis, S3
edge-agent	Streams, inference	Cameras, control-api
notify-worker	Delivery attempts	Redis, email/SMS providers
report-worker	PDF/CSV generation	PG, S3
media-gateway	Signed URLs, retention	S3

Communication: sync REST for control; async queue for notifications/reports; never share DB writes across services without clear ownership.

7. Redis Design (Planned — M3+)

7.1 Roles

Use	Key pattern	TTL
Celery broker	Celery internal	—
Celery result backend	Celery internal	short
JWT refresh blacklist	`jwt:bl:{jti}`	refresh remaining life
Rate limit counters	`rl:{principal}:{window}`	60s
Dashboard cache	`dash:{org_id}:summary`	5–15s
Edge online set	`edge:online:{site_id}`	heartbeat TTL

7.2 Django Settings Sketch

```
CELERY_BROKER_URL = "redis://redis:6379/0"
CELERY_RESULT_BACKEND = "redis://redis:6379/1"
```

```
CACHES = {
"default": {
"BACKEND": "django.core.cache.backends.redis.RedisCache",
"LOCATION": "redis://redis:6379/2",
}
}
```

8. Celery Queues (Planned)

8.1 Queue Map

Queue	Priority	Tasks
`notifications`	High	email/SMS/push for critical alerts
`reports`	Normal	scheduled and on-demand exports
`retention`	Low	delete expired events/evidence
`media`	Normal	thumbnail/clip packaging to S3
`default`	Normal	misc maintenance

8.2 Task Catalog

Task	Trigger	Input	Side effects
`dispatch_alert_notification`	Alert create (critical/warning)	alert_id	NotificationDelivery rows
`generate_incident_report`	API or schedule	org_id, range	PDF in S3 + email link
`purge_expired_events`	Nightly	retention policy	Soft/hard delete + audit
`purge_expired_evidence`	Nightly	retention_until	S3 delete
`recompute_dashboard_cache`	Event ingest debounce	org_id	Redis set

8.3 Worker Deployment

```
celery -A config worker -Q notifications,reports,retention,media,default -c 4
celery -A config beat -l INFO
```

8.4 Alert Notification Flow (Target)

```
sequenceDiagram
participant API
participant Rules
participant Redis
participant Worker
participant Email
API->>Rules: evaluate_alert_rules
Rules->>API: Alert created
API->>Redis: enqueue dispatch_alert_notification
Worker->>Redis: consume
Worker->>Email: send
Worker->>API: update NotificationDelivery
```

9. Object Storage Flow (Planned)

9.1 Buckets

Bucket	Contents	Access
`sv-evidence`	Snapshots/clips	Private; signed GET
`sv-reports`	PDF/CSV exports	Private; signed GET
`sv-models`	Edge model packages	Private; edge pull with key

9.2 Evidence Upload Path

```
Edge captures JPEG/clip on alert-worthy event
-> optional local encrypt
-> POST multipart or pre-signed PUT
-> control plane stores EvidenceArtifact(storage_key, retention_until)
-> UI requests signed URL via API-EVIDENCE-GET
```

Policy: Raw continuous video stays on-prem by default. Only event-linked clips leave the site.

10. RTSP Processing Pipeline (M2 Target)

10.1 Pipeline Stages

```
RTSP URL
-> OpenCV VideoCapture / GStreamer
-> FrameSampler (every Nth frame or time-based)
-> YOLO person/object detect
-> Tracker (ByteTrack) assign track_id
-> ModuleRouter (per camera.enabled_modules)
-> Intrusion (zone polygon test)
-> Face (InsightFace) M3
-> Fire/Smoke/Weapon M5
-> EventBuilder (confidence, bbox, model meta)
-> LocalQueue (retry buffer)
-> HTTPS POST /events/ingest/
-> Heartbeat loop (parallel)
```

10.2 Edge Service Components

Component	File (current)	M2 change
Config	`edge/edge_service/config.py`	Add RTSP list, FPS, GPU flags
Client	`edge/edge_service/client.py`	Keep retry ingest/heartbeat
Inference	`edge/edge_service/inference.py`	Replace SimulatedYOLO with YOLO
Stream	OpenCVStreamAdapter (exists, unused)	Wire into main loop
Main	`edge/edge_service/main.py`	Per-camera worker threads/async

10.3 Frame Sampling Policy

Mode	Rule	Use
Baseline	5–10 FPS decode, infer 2–5 FPS	Person tracking
Alert boost	Temporary 10–15 FPS for 10s after event	Evidence quality

Mode	Rule	Use
Degraded	Drop to 1 FPS on GPU overload	Stability

10.4 Zone Intrusion Logic

```
for detection in persons:
centroid = bbox_center_normalized(detection.bbox)
if point_in_polygon(centroid, zone.polygon_json) and zone.is_restricted:
if schedule_active(now, zone.schedule): # Planned schedule
emit event_type=intrusion severity=critical|warning
```

10.5 Failure Handling

Failure	Behavior
RTSP disconnect	Exponential reconnect; camera status error
API 5xx	Local queue retry with backoff
GPU OOM	Reduce batch/FPS; emit edge health warning
Disk queue full	Drop lowest severity; log critical ops alert

10.6 Current Simulation Path (Implemented)

```
Timer every SIMULATION_INTERVAL
-> SimulatedYOLODetector.detect() # random person_detected
-> build_event_payload(camera_id, detection)
-> ControlPlaneClient.ingest_event
```

This validates control-plane contracts only — not production security.

11. Deployment Architecture

11.1 Local Docker Compose (Implemented)

```
flowchart LR
FE[frontend:3000] --> BE[backend:8000]
Edge[edge] --> BE
BE --> DB[(postgres:5432)]
```

Service	Image role	Port
db	postgres:15-alpine	5432
backend	Django/Gunicorn	8000
frontend	Nginx + React build	3000
edge	Python edge worker	—

Healthchecks: Postgres `pg_isready`; backend HTTP `/api/v1/health/`.

11.2 Staging / Production (Planned)

```
flowchart TB
Users --> LB[LoadBalancer_TLS]
LB --> Nginx
Nginx --> APIPods[DjangoPods]
Nginx --> WebPods[StaticFrontend]
```

```
APIPods --> PG[(ManagedPostgreSQL)]
APIPods --> Redis[(Redis)]
Workers[CeleryPods] --> Redis
Workers --> S3
APIPods --> S3
EdgeSites[CustomerEdgeAgents] -->|egress_HTTPS| LB
```

11.3 Kubernetes Workloads (Planned)

Workload	Replicas	Notes
`api` Deployment	2–N	HPA on CPU / RPS
`worker` Deployment	1–N	One deployment per queue group optional
`beat` Deployment	1	Singleton
`frontend` Deployment	2	CDN preferred
Ingress	1	cert-manager TLS
NetworkPolicy	—	Deny DB from non-api

11.4 Environment Matrix

Env	Compose	K8s	Real cameras	Secrets
Local	Yes	No	Optional	`.env`
CI	Ephemeral	No	No	GHA secrets
Staging	Optional	Yes	Lab RTSP	Vault/ESO
Production	No	Yes	Customer	Vault/ESO

11.5 Nginx Responsibilities

Path	Upstream
`/`	Frontend static
`/api/`	Django API
`/api/docs/`	Spectacular Swagger
`/healthz`	Local Nginx health

Production adds: TLS, HSTS, request size limits, edge ingest rate limits.

12. Monitoring and Logging (Target)

Signal	Tool	Alert example
API latency/error rate	Prometheus + Grafana	p95 > 1s or 5xx > 1%
Celery queue depth	Redis metrics	notifications depth > 1000
Edge heartbeat age	Custom metric	last_seen > 2 min

Signal	Tool	Alert example
Disk / GPU	Node exporter / DCGM	GPU mem > 90%
App logs	ELK/OpenSearch	ERROR spikes
Audit trail	Postgres audit_auditlog	Missing writes (integrity)

Current: Only `GET /api/v1/health/` is Implemented.

13. Security Design Highlights

Control	Status
JWT short-lived access tokens	Implemented
Edge key hashed at rest	Implemented
Tenant query scoping	Implemented
Role gates on writes	Implemented
TLS everywhere	Planned (prod)
Rate limiting	Planned
Constant-time key compare	Planned
Device-bound ingest only	Remediation required
Audit auto-logging	Remediation required
Secrets in Vault	Planned

14. Capacity Model

14.1 Edge Node Sizing

Profile	Cameras	GPU	Concurrent modules
Small	4	Jetson Orin Nano	detection + tracking
Medium	8–12	RTX 3060	+ face
Large	12–16	RTX 4070+	+ threat modules

14.2 Control Plane Sizing (per 100 sites / 1k cameras)

Component	Baseline
API pods	4 × 2 vCPU / 4 GB
Workers	2 × 2 vCPU / 4 GB
PostgreSQL	8 vCPU / 32 GB / SSD
Redis	2 GB

Component	Baseline
Object storage	Elastic

15. Development Sequence Mapping

Milestone	System design focus
M1	Django apps, JWT, RBAC, tenancy, dashboard APIs — **Implemented**
M2	RTSP, OpenCV, YOLO, tracker, real ingest, zone intrusion
M3	InsightFace, enrollment, attendance tables + APIs
M4	Unknown person, visitors, intrusion hardening
M5	Fire/smoke/weapon models + critical notification path
M6	Reports, Celery notifications, mobile/PWA, analytics

16. Engineering Checklists

16.1 Before M2 Coding Starts

- [ ] Camera has valid RTSP and enabled_modules
- [ ] Edge device registered; API key stored on node
- [ ] Alert rules for `person_detected` / `intrusion` seeded
- [ ] Volume 4 integrity triggers scheduled (DB-FIX-*)
- [ ] Volume 5 ingest contract tests green

16.2 Definition of Done — M2 Vertical Slice

- [ ] One live RTSP stream decoded on edge
- [ ] YOLO detections produce real events (not random)
- [ ] Track IDs stable across frames for short windows
- [ ] Restricted zone intrusion creates critical alert
- [ ] Operator reviews event and acknowledges alert in UI

17. Traceability

Design Area	Volumes	Status
Django apps	Vol 4, Vol 5	Implemented
Edge pipeline	Vol 6 AI	Partial / M2
Celery/Redis	Vol 8 DevOps	Planned
Storage	Vol 4 Evidence	Planned
Deployment	Vol 8	Compose Implemented; K8s Planned

SentinelVision AI

Volume 04

Database Design

Suite Version	1.0.0
Classification	Internal / Confidential
Generated	2026-07-17 07:23 UTC

volume: 04

title: Database Design

status: engineering-ready

version: 2.0.0

> **SentinelVision AI — Volume 4**

> Status: Implemented | Planned | Future

1. Document Control

Field	Value
Document ID	DOC-DB-004
Version	2.0.0
Status	Engineering Ready
Primary RDBMS	PostgreSQL 15+
Local fallback	SQLite (dev only)
ORM	Django 5.x migrations

2. Design Principles

1. UUID primary keys for all domain tables (API-stable identifiers).
2. Tenant isolation via `organization_id` or join through `site` → `organization`.
3. Soft operational flags (`is_active`) preferred over hard deletes for config entities.
4. JSONB for flexible metadata/payloads; typed columns for queryable fields.
5. Indexes for time-series event/alert queries.
6. Additive migrations only in production without approved downtime window.

3. Logical ER Diagram

```
erDiagram
auth_user ||--o{ tenancy_usermembership : has
tenancy_organization ||--o{ tenancy_usermembership : has
tenancy_organization ||--o{ sites_site : owns
tenancy_organization ||--o{ alerts_alertrule : defines
tenancy_organization ||--o{ alerts_alert : owns
tenancy_organization ||--o{ audit_auditlog : scopes
sites_site ||--o{ sites_zone : contains
sites_site ||--o{ cameras_camera : mounts
sites_site ||--o{ cameras_edgedevice : hosts
sites_zone ||--o{ cameras_camera : optional
cameras_camera ||--o{ events_detectionevent : generates
sites_zone ||--o{ events_detectionevent : optional
events_detectionevent ||--o{ alerts_alert : triggers
alerts_alertrule ||--o{ alerts_alert : matches
auth_user ||--o{ events_detectionevent : reviews
auth_user ||--o{ alerts_alert : acknowledges
```

4. Schema Overview (Implemented)

Django App	Table	Model
tenancy	`tenancy_organization`	Organization
tenancy	`tenancy_usermembership`	UserMembership
sites	`sites_site`	Site
sites	`sites_zone`	Zone
cameras	`cameras_camera`	Camera
cameras	`cameras_edgedevice`	EdgeDevice
events	`events_detectionevent`	DetectionEvent
alerts	`alerts_alertrule`	AlertRule
alerts	`alerts_alert`	Alert
audit	`audit_auditlog`	AuditLog

Django also uses `auth_user`, `auth_group`, `django_session`, `django_migrations`, etc.

5. DDL — Implemented Tables (PostgreSQL)

> Generated to match current Django models. Apply via `python manage.py migrate` in production; DDL below is the engineering reference.

5.1 tenancy_organization

```
CREATE TABLE tenancy_organization (
id UUID PRIMARY KEY,
name VARCHAR(255) NOT NULL,
slug VARCHAR(100) NOT NULL,
settings JSONB NOT NULL DEFAULT '{}'::jsonb,
is_active BOOLEAN NOT NULL DEFAULT TRUE,
created_at TIMESTAMPTZ NOT NULL,
updated_at TIMESTAMPTZ NOT NULL,
CONSTRAINT tenancy_organization_slug_key UNIQUE (slug)
);
CREATE INDEX tenancy_organization_slug_like
ON tenancy_organization (slug varchar_pattern_ops);
```

Column	Type	Null	Default	Notes
id	UUID	NO	uuid4	PK
name	VARCHAR(255)	NO	—	Display name
slug	VARCHAR(100)	NO	—	Unique URL/key
settings	JSONB	NO	{}	Retention, feature flags
is_active	BOOLEAN	NO	TRUE	Soft disable
created_at	TIMESTAMPTZ	NO	now	
updated_at	TIMESTAMPTZ	NO	now	

5.2 tenancy_usermembership

```
CREATE TABLE tenancy_usermembership (
```

```
id UUID PRIMARY KEY,
role VARCHAR(20) NOT NULL DEFAULT 'viewer',
created_at TIMESTAMPTZ NOT NULL,
updated_at TIMESTAMPTZ NOT NULL,
organization_id UUID NOT NULL REFERENCES tenancy_organization(id) ON DELETE CASCADE,
user_id INTEGER NOT NULL REFERENCES auth_user(id) ON DELETE CASCADE,
CONSTRAINT tenancy_usermembership_user_org_uniq UNIQUE (user_id, organization_id),
CONSTRAINT tenancy_usermembership_role_chk CHECK (role IN ('owner','admin','operator','viewer'))
);
CREATE INDEX tenancy_usermembership_org_idx ON tenancy_usermembership (organization_id);
CREATE INDEX tenancy_usermembership_user_idx ON tenancy_usermembership (user_id);
```

5.3 sites_site

```
CREATE TABLE sites_site (
id UUID PRIMARY KEY,
name VARCHAR(255) NOT NULL,
address TEXT NOT NULL DEFAULT '',
timezone VARCHAR(64) NOT NULL DEFAULT 'UTC',
is_active BOOLEAN NOT NULL DEFAULT TRUE,
created_at TIMESTAMPTZ NOT NULL,
updated_at TIMESTAMPTZ NOT NULL,
organization_id UUID NOT NULL REFERENCES tenancy_organization(id) ON DELETE CASCADE,
CONSTRAINT sites_site_org_name_uniq UNIQUE (organization_id, name)
);
CREATE INDEX sites_site_org_idx ON sites_site (organization_id);
```

5.4 sites_zone

```
CREATE TABLE sites_zone (
id UUID PRIMARY KEY,
name VARCHAR(255) NOT NULL,
polygon_json JSONB NOT NULL DEFAULT '[]'::jsonb,
is_restricted BOOLEAN NOT NULL DEFAULT FALSE,
is_active BOOLEAN NOT NULL DEFAULT TRUE,
created_at TIMESTAMPTZ NOT NULL,
updated_at TIMESTAMPTZ NOT NULL,
site_id UUID NOT NULL REFERENCES sites_site(id) ON DELETE CASCADE,
CONSTRAINT sites_zone_site_name_uniq UNIQUE (site_id, name)
);
CREATE INDEX sites_zone_site_idx ON sites_zone (site_id);
```

polygon_json example:

```
[[0.1, 0.1], [0.9, 0.1], [0.9, 0.9], [0.1, 0.9]]
```

Coordinates are normalized image fractions (0.0–1.0).

5.5 cameras_camera

```
CREATE TABLE cameras_camera (
id UUID PRIMARY KEY,
name VARCHAR(255) NOT NULL,
rtsp_url VARCHAR(512) NOT NULL DEFAULT '',
status VARCHAR(20) NOT NULL DEFAULT 'offline',
enabled_modules JSONB NOT NULL DEFAULT '[]'::jsonb,
last_seen_at TIMESTAMPTZ NULL,
metadata JSONB NOT NULL DEFAULT '{}'::jsonb,
is_active BOOLEAN NOT NULL DEFAULT TRUE,
created_at TIMESTAMPTZ NOT NULL,
updated_at TIMESTAMPTZ NOT NULL,
site_id UUID NOT NULL REFERENCES sites_site(id) ON DELETE CASCADE,
zone_id UUID NULL REFERENCES sites_zone(id) ON DELETE SET NULL,
CONSTRAINT cameras_camera_site_name_uniq UNIQUE (site_id, name),
CONSTRAINT cameras_camera_status_chk CHECK (
status IN ('online','offline','error','maintenance')
)
);
CREATE INDEX cameras_camera_site_idx ON cameras_camera (site_id);
CREATE INDEX cameras_camera_zone_idx ON cameras_camera (zone_id);
CREATE INDEX cameras_camera_status_idx ON cameras_camera (status);
```

5.6 cameras_edgedevice

```
CREATE TABLE cameras_edgedevice (
id UUID PRIMARY KEY,
name VARCHAR(255) NOT NULL,
api_key_hash VARCHAR(128) NOT NULL,
api_key_prefix VARCHAR(8) NOT NULL DEFAULT '',
last_seen_at TIMESTAMPTZ NULL,
status VARCHAR(20) NOT NULL DEFAULT 'offline',
version VARCHAR(64) NOT NULL DEFAULT '',
metadata JSONB NOT NULL DEFAULT '{}'::jsonb,
is_active BOOLEAN NOT NULL DEFAULT TRUE,
created_at TIMESTAMPTZ NOT NULL,
updated_at TIMESTAMPTZ NOT NULL,
site_id UUID NOT NULL REFERENCES sites_site(id) ON DELETE CASCADE,
CONSTRAINT cameras_edgedevice_site_name_uniq UNIQUE (site_id, name)
);
CREATE INDEX cameras_edgedevice_site_idx ON cameras_edgedevice (site_id);
CREATE UNIQUE INDEX cameras_edgedevice_api_key_hash_uniq ON cameras_edgedevice (api_key_hash);
CREATE INDEX cameras_edgedevice_last_seen_idx ON cameras_edgedevice (last_seen_at DESC);
```

Security: Never store plaintext API keys. Hash with SHA-256 before insert.

5.7 events_detectionevent

```
CREATE TABLE events_detectionevent (
id UUID PRIMARY KEY,
event_type VARCHAR(32) NOT NULL,
severity VARCHAR(16) NOT NULL DEFAULT 'info',
confidence DOUBLE PRECISION NOT NULL DEFAULT 0.0,
model_name VARCHAR(128) NOT NULL DEFAULT '',
model_version VARCHAR(64) NOT NULL DEFAULT '',
payload JSONB NOT NULL DEFAULT '{}'::jsonb,
review_status VARCHAR(20) NOT NULL DEFAULT 'pending',
reviewed_at TIMESTAMPTZ NULL,
retention_until TIMESTAMPTZ NULL,
edge_event_id VARCHAR(128) NOT NULL DEFAULT '',
created_at TIMESTAMPTZ NOT NULL,
updated_at TIMESTAMPTZ NOT NULL,
camera_id UUID NOT NULL REFERENCES cameras_camera(id) ON DELETE CASCADE,
zone_id UUID NULL REFERENCES sites_zone(id) ON DELETE SET NULL,
reviewed_by_id INTEGER NULL REFERENCES auth_user(id) ON DELETE SET NULL,
CONSTRAINT events_detectionevent_type_chk CHECK (
event_type IN (
'person_detected','face_recognized','unknown_person','intrusion',
'activity','fire_smoke','weapon','attendance','visitor'
)
),
CONSTRAINT events_detectionevent_severity_chk CHECK (
severity IN ('info','warning','critical')
),
CONSTRAINT events_detectionevent_review_chk CHECK (
review_status IN ('pending','confirmed','false_positive','escalated')
),
CONSTRAINT events_detectionevent_confidence_chk CHECK (
confidence >= 0.0 AND confidence <= 1.0
)
);
CREATE INDEX events_detectionevent_camera_type_idx
ON events_detectionevent (camera_id, event_type);
CREATE INDEX events_detectionevent_created_idx
ON events_detectionevent (created_at DESC);
CREATE INDEX events_detectionevent_edge_event_id_idx
ON events_detectionevent (edge_event_id);
CREATE INDEX events_detectionevent_review_idx
ON events_detectionevent (review_status);
```

Planned hardening indexes/constraints:

```
-- Denormalized tenant key for faster isolation & idempotency
ALTER TABLE events_detectionevent
ADD COLUMN organization_id UUID NULL REFERENCES tenancy_organization(id);
CREATE UNIQUE INDEX events_detectionevent_org_edge_uniq
ON events_detectionevent (organization_id, edge_event_id)
```

```
WHERE edge_event_id <> '';
CREATE INDEX events_detectionevent_org_created_idx
ON events_detectionevent (organization_id, created_at DESC);
```

5.8 alerts_alertrule

```
CREATE TABLE alerts_alertrule (
id UUID PRIMARY KEY,
name VARCHAR(255) NOT NULL,
event_type VARCHAR(32) NOT NULL,
min_severity VARCHAR(16) NOT NULL DEFAULT 'warning',
min_confidence DOUBLE PRECISION NOT NULL DEFAULT 0.5,
is_active BOOLEAN NOT NULL DEFAULT TRUE,
notify_email BOOLEAN NOT NULL DEFAULT TRUE,
conditions JSONB NOT NULL DEFAULT '{}'::jsonb,
created_at TIMESTAMPTZ NOT NULL,
updated_at TIMESTAMPTZ NOT NULL,
organization_id UUID NOT NULL REFERENCES tenancy_organization(id) ON DELETE CASCADE,
CONSTRAINT alerts_alertrule_confidence_chk CHECK (
min_confidence >= 0.0 AND min_confidence <= 1.0
)
);
CREATE INDEX alerts_alertrule_org_idx ON alerts_alertrule (organization_id);
CREATE INDEX alerts_alertrule_type_active_idx
ON alerts_alertrule (organization_id, event_type, is_active);
```

5.9 alerts_alert

```
CREATE TABLE alerts_alert (
id UUID PRIMARY KEY,
title VARCHAR(512) NOT NULL,
message TEXT NOT NULL DEFAULT '',
severity VARCHAR(16) NOT NULL,
status VARCHAR(20) NOT NULL DEFAULT 'open',
acknowledged_at TIMESTAMPTZ NULL,
resolved_at TIMESTAMPTZ NULL,
created_at TIMESTAMPTZ NOT NULL,
updated_at TIMESTAMPTZ NOT NULL,
organization_id UUID NOT NULL REFERENCES tenancy_organization(id) ON DELETE CASCADE,
event_id UUID NOT NULL REFERENCES events_detectionevent(id) ON DELETE CASCADE,
rule_id UUID NULL REFERENCES alerts_alertrule(id) ON DELETE SET NULL,
acknowledged_by_id INTEGER NULL REFERENCES auth_user(id) ON DELETE SET NULL,
CONSTRAINT alerts_alert_status_chk CHECK (
status IN ('open','acknowledged','resolved','dismissed')
),
CONSTRAINT alerts_alert_severity_chk CHECK (
severity IN ('info','warning','critical')
)
);
CREATE INDEX alerts_alert_org_status_created_idx
ON alerts_alert (organization_id, status, created_at DESC);
CREATE INDEX alerts_alert_event_idx ON alerts_alert (event_id);
```

5.10 audit_auditlog

```
CREATE TABLE audit_auditlog (
id UUID PRIMARY KEY,
action VARCHAR(64) NOT NULL,
resource_type VARCHAR(64) NOT NULL,
resource_id VARCHAR(64) NOT NULL DEFAULT '',
ip_address INET NULL,
details JSONB NOT NULL DEFAULT '{}'::jsonb,
created_at TIMESTAMPTZ NOT NULL,
updated_at TIMESTAMPTZ NOT NULL,
organization_id UUID NULL REFERENCES tenancy_organization(id) ON DELETE CASCADE,
actor_id INTEGER NULL REFERENCES auth_user(id) ON DELETE SET NULL
);
CREATE INDEX audit_auditlog_org_created_idx
ON audit_auditlog (organization_id, created_at DESC);
CREATE INDEX audit_auditlog_action_idx ON audit_auditlog (action);
```

```
CREATE INDEX audit_auditlog_resource_idx
ON audit_auditlog (resource_type, resource_id);
```

Immutability (Planned): Revoke UPDATE/DELETE from application role; allow INSERT only.

```
-- Planned: application role privileges
REVOKE UPDATE, DELETE ON audit_auditlog FROM sentinel_app;
GRANT INSERT, SELECT ON audit_auditlog TO sentinel_app;
```

6. Triggers (Planned + Recommended)

6.1 updated_at maintenance

Django currently sets `updated_at` in application code (`auto_now`). Optional DB trigger for defense in depth:

```
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
NEW.updated_at = NOW();
RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_cameras_camera_updated
BEFORE UPDATE ON cameras_camera
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
-- Repeat for other timestamped tables as needed
```

6.2 Zone belongs to camera site

```
CREATE OR REPLACE FUNCTION enforce_camera_zone_same_site()
RETURNS TRIGGER AS $$
BEGIN
IF NEW.zone_id IS NULL THEN
RETURN NEW;
END IF;
IF NOT EXISTS (
SELECT 1 FROM sites_zone z
WHERE z.id = NEW.zone_id AND z.site_id = NEW.site_id
) THEN
RAISE EXCEPTION 'zone_id must belong to the same site as camera';
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_camera_zone_site
BEFORE INSERT OR UPDATE OF zone_id, site_id ON cameras_camera
FOR EACH ROW EXECUTE FUNCTION enforce_camera_zone_same_site();
```

6.3 Event zone same site as camera

```
CREATE OR REPLACE FUNCTION enforce_event_zone_same_site()
RETURNS TRIGGER AS $$
DECLARE
cam_site UUID;
BEGIN
IF NEW.zone_id IS NULL THEN
RETURN NEW;
END IF;
SELECT site_id INTO cam_site FROM cameras_camera WHERE id = NEW.camera_id;
IF NOT EXISTS (
SELECT 1 FROM sites_zone z WHERE z.id = NEW.zone_id AND z.site_id = cam_site
) THEN
RAISE EXCEPTION 'event zone must belong to camera site';
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_event_zone_site
BEFORE INSERT OR UPDATE OF zone_id, camera_id ON events_detectionevent
```

```
FOR EACH ROW EXECUTE FUNCTION enforce_event_zone_same_site();
```

6.4 Alert organization matches event organization

```
CREATE OR REPLACE FUNCTION enforce_alert_org_matches_event()
RETURNS TRIGGER AS $$
DECLARE
event_org UUID;
BEGIN
SELECT s.organization_id INTO event_org
FROM events_detectionevent e
JOIN cameras_camera c ON c.id = e.camera_id
JOIN sites_site s ON s.id = c.site_id
WHERE e.id = NEW.event_id;
IF event_org IS DISTINCT FROM NEW.organization_id THEN
RAISE EXCEPTION 'alert.organization_id must match event camera organization';
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_alert_org_event
BEFORE INSERT OR UPDATE OF organization_id, event_id ON alerts_alert
FOR EACH ROW EXECUTE FUNCTION enforce_alert_org_matches_event();
```

7. Planned Tables (M3–M6)

7.1 identity_personprofile

```
CREATE TABLE identity_personprofile (
id UUID PRIMARY KEY,
organization_id UUID NOT NULL REFERENCES tenancy_organization(id) ON DELETE CASCADE,
external_id VARCHAR(64) NOT NULL DEFAULT '',
full_name VARCHAR(255) NOT NULL,
person_type VARCHAR(20) NOT NULL, -- employee | visitor | contractor
email VARCHAR(254) NOT NULL DEFAULT '',
is_active BOOLEAN NOT NULL DEFAULT TRUE,
metadata JSONB NOT NULL DEFAULT '{}'::jsonb,
created_at TIMESTAMPTZ NOT NULL,
updated_at TIMESTAMPTZ NOT NULL,
CONSTRAINT identity_personprofile_type_chk CHECK (
person_type IN ('employee','visitor','contractor')
),
UNIQUE (organization_id, external_id)
);
```

7.2 identity_biometricconsent

```
CREATE TABLE identity_biometricconsent (
id UUID PRIMARY KEY,
organization_id UUID NOT NULL REFERENCES tenancy_organization(id) ON DELETE CASCADE,
person_profile_id UUID NOT NULL REFERENCES identity_personprofile(id) ON DELETE CASCADE,
consent_type VARCHAR(32) NOT NULL, -- enrollment | retention
granted_at TIMESTAMPTZ NOT NULL,
revoked_at TIMESTAMPTZ NULL,
evidence_url TEXT NOT NULL DEFAULT '',
created_at TIMESTAMPTZ NOT NULL,
updated_at TIMESTAMPTZ NOT NULL
);
```

7.3 identity_faceembedding

```
CREATE TABLE identity_faceembedding (
id UUID PRIMARY KEY,
organization_id UUID NOT NULL REFERENCES tenancy_organization(id) ON DELETE CASCADE,
person_profile_id UUID NOT NULL REFERENCES identity_personprofile(id) ON DELETE CASCADE,
consent_id UUID NOT NULL REFERENCES identity_biometricconsent(id),
```

```
model_name VARCHAR(128) NOT NULL,
model_version VARCHAR(64) NOT NULL,
embedding BYTEA NOT NULL, -- or vector extension
quality_score DOUBLE PRECISION NOT NULL DEFAULT 0.0,
is_active BOOLEAN NOT NULL DEFAULT TRUE,
created_at TIMESTAMPTZ NOT NULL,
updated_at TIMESTAMPTZ NOT NULL
);
-- Optional pgvector:
-- ALTER TABLE identity_faceembedding ADD COLUMN embedding_vec vector(512);
```

7.4 visitor_registration

```
CREATE TABLE visitor_registration (
id UUID PRIMARY KEY,
organization_id UUID NOT NULL REFERENCES tenancy_organization(id) ON DELETE CASCADE,
site_id UUID NOT NULL REFERENCES sites_site(id) ON DELETE CASCADE,
person_profile_id UUID NULL REFERENCES identity_personprofile(id),
host_name VARCHAR(255) NOT NULL,
purpose TEXT NOT NULL DEFAULT '',
valid_from TIMESTAMPTZ NOT NULL,
valid_until TIMESTAMPTZ NOT NULL,
status VARCHAR(20) NOT NULL DEFAULT 'scheduled',
created_at TIMESTAMPTZ NOT NULL,
updated_at TIMESTAMPTZ NOT NULL,
CONSTRAINT visitor_registration_status_chk CHECK (
status IN ('scheduled','checked_in','checked_out','cancelled','expired')
)
);
```

7.5 attendance_record

```
CREATE TABLE attendance_record (
id UUID PRIMARY KEY,
organization_id UUID NOT NULL REFERENCES tenancy_organization(id) ON DELETE CASCADE,
person_profile_id UUID NOT NULL REFERENCES identity_personprofile(id) ON DELETE CASCADE,
site_id UUID NOT NULL REFERENCES sites_site(id),
camera_id UUID NULL REFERENCES cameras_camera(id),
event_id UUID NULL REFERENCES events_detectionevent(id),
direction VARCHAR(10) NOT NULL, -- in | out
recorded_at TIMESTAMPTZ NOT NULL,
confidence DOUBLE PRECISION NOT NULL DEFAULT 0.0,
created_at TIMESTAMPTZ NOT NULL,
updated_at TIMESTAMPTZ NOT NULL,
CONSTRAINT attendance_direction_chk CHECK (direction IN ('in','out'))
);
CREATE INDEX attendance_person_day_idx
ON attendance_record (organization_id, person_profile_id, recorded_at DESC);
```

7.6 evidence_artifact

```
CREATE TABLE evidence_artifact (
id UUID PRIMARY KEY,
organization_id UUID NOT NULL REFERENCES tenancy_organization(id) ON DELETE CASCADE,
event_id UUID NOT NULL REFERENCES events_detectionevent(id) ON DELETE CASCADE,
storage_backend VARCHAR(32) NOT NULL DEFAULT 's3',
storage_key TEXT NOT NULL,
media_type VARCHAR(64) NOT NULL,
byte_size BIGINT NOT NULL DEFAULT 0,
retention_until TIMESTAMPTZ NOT NULL,
checksum_sha256 VARCHAR(64) NOT NULL DEFAULT '',
created_at TIMESTAMPTZ NOT NULL,
updated_at TIMESTAMPTZ NOT NULL
);
CREATE INDEX evidence_artifact_event_idx ON evidence_artifact (event_id);
CREATE INDEX evidence_artifact_retention_idx ON evidence_artifact (retention_until);
```

7.7 notifications_delivery

```
CREATE TABLE notifications_delivery (
id UUID PRIMARY KEY,
organization_id UUID NOT NULL REFERENCES tenancy_organization(id) ON DELETE CASCADE,
alert_id UUID NOT NULL REFERENCES alerts_alert(id) ON DELETE CASCADE,
channel VARCHAR(20) NOT NULL, -- email | sms | push | webhook
recipient VARCHAR(255) NOT NULL,
status VARCHAR(20) NOT NULL DEFAULT 'pending',
attempts INTEGER NOT NULL DEFAULT 0,
last_error TEXT NOT NULL DEFAULT '',
sent_at TIMESTAMPTZ NULL,
created_at TIMESTAMPTZ NOT NULL,
updated_at TIMESTAMPTZ NOT NULL,
CONSTRAINT notifications_channel_chk CHECK (
channel IN ('email','sms','push','webhook')
),
CONSTRAINT notifications_status_chk CHECK (
status IN ('pending','sent','failed','cancelled')
)
);
```

7.8 ai_modelregistry

```
CREATE TABLE ai_modelregistry (
id UUID PRIMARY KEY,
module_key VARCHAR(64) NOT NULL,
model_name VARCHAR(128) NOT NULL,
model_version VARCHAR(64) NOT NULL,
artifact_uri TEXT NOT NULL,
metrics JSONB NOT NULL DEFAULT '{}'::jsonb,
is_production BOOLEAN NOT NULL DEFAULT FALSE,
created_at TIMESTAMPTZ NOT NULL,
updated_at TIMESTAMPTZ NOT NULL,
UNIQUE (module_key, model_name, model_version)
);
```

8. Index Strategy Summary

Table	Index	Purpose
events_detectionevent	(camera_id, event_type)	Camera event type filter
events_detectionevent	(created_at DESC)	Timeline queries
events_detectionevent	(edge_event_id)	Idempotency lookup
alerts_alert	(organization_id, status, created_at DESC)	Operator alert queue
audit_auditlog	(organization_id, created_at DESC)	Compliance export
cameras_edgedevice	UNIQUE(api_key_hash)	Auth lookup
attendance_record	(org, person, recorded_at)	Daily attendance

9. Relationships Matrix

Parent	Child	FK	On Delete
Organization	Site	organization_id	CASCADE
Organization	UserMembership	organization_id	CASCADE
Site	Zone	site_id	CASCADE

Parent	Child	FK	On Delete
Site	Camera	site_id	CASCADE
Site	EdgeDevice	site_id	CASCADE
Zone	Camera	zone_id	SET NULL
Camera	DetectionEvent	camera_id	CASCADE
DetectionEvent	Alert	event_id	CASCADE
Organization	Alert	organization_id	CASCADE
AlertRule	Alert	rule_id	SET NULL

10. Migration Strategy

Phase	Django Migration Focus
M0/M1	Current 10 domain tables (done)
M2	organization_id on events; uniqueness; integrity triggers
M3	identity_, *visitor_*, attendance_*
M4	evidence_artifact; ai_modelregistry
M5	notifications_delivery
All	Squash only on greenfield; never squash production history

Rollback policy: Prefer forward-fix migrations. Destructive rollbacks require snapshot restore.

11. Seed Data Reference

`python manage.py seed_dev_data` creates:

Entity	Example
Organization	ACME Security Corp / `acme-security`
Site	Headquarters
Zone	Server Room (restricted)
Camera	Lobby Camera 01
EdgeDevice	Edge Node HQ-01 (dev API key hash)
Users	admin/admin123 (owner), operator/operator123
AlertRules	Intrusion Critical, Person Detected Warning

12. Data Integrity Remediation Backlog

ID	Issue	Fix
DB-FIX-01	Global edge_event_id lookup	Unique (organization_id, edge_event_id)
DB-FIX-02	Cross-site zone on camera	Trigger + serializer validation
DB-FIX-03	Event zone mismatch	Trigger enforce_event_zone_same_site
DB-FIX-04	Alert org mismatch	Trigger enforce_alert_org_matches_event
DB-FIX-05	Audit not append-only	DB privileges + no update API
DB-FIX-06	Missing org index on events	Denormalize organization_id

13. Capacity Guidance

Entity	1 year estimate (mid tenant)	Notes
DetectionEvent	50M–200M rows	Partition by month Planned
Alert	1M–10M	Archive resolved after 180d
Evidence	1–20 TB	Object storage, not Postgres
AuditLog	5M–50M	Partition by month Planned

Partitioning (Future):

```
-- Example monthly partition parent
CREATE TABLE events_detectionevent_y2026m07
PARTITION OF events_detectionevent
FOR VALUES FROM ('2026-07-01') TO ('2026-08-01');
```

14. Traceability

DB Entity	API	FR	Status
DB-Organization	API-ORG-LIST	BR-TEN-01	Implemented
DB-Camera	API-CAM-LIST	FR-CAM-01	Implemented
DB-DetectionEvent	API-EVT-INGEST	UC-EVT-01	Implemented
DB-Alert	API-ALERT-ACK	FR-ALERT-04	Implemented
DB-PersonProfile	API-PERSON-ENROLL	US-10	Planned
DB-EvidenceArtifact	API-EVIDENCE-GET	US-02	Planned

SentinelVision AI

Volume 05

REST API Documentation

Suite Version	1.0.0
Classification	Internal / Confidential
Generated	2026-07-17 07:23 UTC

volume: 05

title: REST API Documentation

status: engineering-ready

version: 2.0.0

> **SentinelVision AI — Volume 5**

> Status: Implemented | Planned | Future

1. Document Control

Field	Value
Document ID	DOC-API-005
Version	2.0.0
Status	Engineering Ready
Base URL (local)	`http://localhost:8000`
API Prefix	`/api/v1/`
OpenAPI Schema	`GET /api/schema/`
Swagger UI	`GET /api/docs/`

2. API Conventions

Convention	Specification
Protocol	HTTP/1.1 (TLS 1.2+ in production)
Content-Type	`application/json`
Charset	UTF-8
Timestamps	ISO 8601 UTC (`2026-07-17T07:00:00.000000Z`)
Primary keys	UUID v4 in path and response
Pagination	PageNumberPagination, `PAGE_SIZE=25` (Implemented); cursor Planned
Filtering	`django-filter` query params where documented
Ordering	`?ordering=field` or `?ordering=-field`
Search	`?search=term` where documented

2.1 Authentication Methods

Method	Header	Used By	Status
JWT Bearer	`Authorization: Bearer <access_token>`	Dashboard users	Implemented

Method	Header	Used By	Status
Edge API Key	`X-Edge-API-Key: <raw_key>`	Edge workers	Implemented
Edge Alt	`Authorization: Edge <raw_key>`	Edge workers	Implemented

2.2 JWT Token Policy

Token	Lifetime	Notes
Access	15 minutes	Used for API calls
Refresh	7 days	`ROTATE_REFRESH_TOKENS=True`; blacklist Planned

2.3 Role Permissions Matrix

Capability	Owner	Admin	Operator	Viewer
Read org/sites/cameras/events/alerts	Yes	Yes	Yes	Yes
Create/update/delete sites, zones, cameras	Yes	Yes	No	No
Create edge devices	Yes	Yes	No	No
Create alert rules	Yes	Yes	No	No
Review events	Yes	Yes	Yes	No
Acknowledge/resolve alerts	Yes	Yes	Yes	No
List audit logs	Yes	Yes	Yes	Yes

2.4 Standard Error Responses

Current (DRF default) for most endpoints:

```
{
"detail": "Authentication credentials were not provided."
}
```

```
{
"field_name": ["This field is required."]
}
```

Target envelope (Planned — use for new endpoints):

```
{
"error": {
"code": "VALIDATION_ERROR",
"message": "Human-readable summary",
"details": {"field": ["reason"]}
}
}
```

HTTP	Code (Planned)	Meaning
400	VALIDATION_ERROR	Invalid request body/query
401	UNAUTHENTICATED	Missing or invalid credentials

HTTP	Code (Planned)	Meaning
403	FORBIDDEN	Authenticated but insufficient role/tenant
404	NOT_FOUND	Resource does not exist or not visible
409	CONFLICT	Duplicate / state conflict
429	RATE_LIMITED	Too many requests (Planned)
500	INTERNAL_ERROR	Unexpected server error

2.5 Pagination Envelope

```
{
"count": 100,
"next": "http://localhost:8000/api/v1/events/?page=2",
"previous": null,
"results": []
}
```

3. Health & Schema

API-HEALTH — Health Check

Field	Value
Method / URL	`GET /api/v1/health/`
Auth	None
Permissions	Public
Status	Implemented

Response `200`**:**

```
{
"status": "healthy",
"service": "sentinelvision-api",
"timestamp": "2026-07-17T07:00:00.123456+00:00"
}
```

API-SCHEMA — OpenAPI Schema

Field	Value
Method / URL	`GET /api/schema/`
Auth	None
Format	OpenAPI 3.0 YAML/JSON
Status	Implemented via `drf-spectacular`

API-DOCS — Swagger UI

Field	Value
Method / URL	`GET /api/docs/`
Auth	None
Status	Implemented

4. Authentication Endpoints

API-AUTH-TOKEN — Obtain JWT Pair

Field	Value
Method / URL	`POST /api/v1/auth/token/`
Auth	None
Status	Implemented

Request:

```
{
"username": "admin",
"password": "admin123"
}
```

Validation:

Field	Rules
username	Required, string
password	Required, string

Response `200`:

```
{
"access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
"refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Errors:

Status	Condition
400	Missing fields
401	Invalid credentials

API-AUTH-REFRESH — Refresh Access Token

Field	Value
Method / URL	`POST /api/v1/auth/token/refresh/`
Auth	None (body carries refresh)
Status	Implemented

Request:

```
{
```

```
"refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Response `200`:

```
{
"access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
"refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

5. Tenancy Endpoints

API-ORG-LIST — List Organizations

Field	Value
Method / URL	`GET /api/v1/organizations/`
Auth	JWT
Permissions	Authenticated; scoped to memberships
Status	Implemented

Response `200` (paginated):

```
{
"count": 1,
"next": null,
"previous": null,
"results": [
{
"id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
"name": "ACME Security Corp",
"slug": "acme-security",
"settings": {"retention_days": 90},
"is_active": true,
"created_at": "2026-07-17T06:00:00Z",
"updated_at": "2026-07-17T06:00:00Z"
}
]
}
```

API-MEMBERSHIP-ME — Current User Memberships

Field	Value
Method / URL	`GET /api/v1/memberships/me/`
Auth	JWT
Status	Implemented

Response `200`:

```
[
{
"organization": {
"id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
"name": "ACME Security Corp",
"slug": "acme-security",
"settings": {"retention_days": 90},
"is_active": true,
"created_at": "2026-07-17T06:00:00Z",
"updated_at": "2026-07-17T06:00:00Z"
},
"role": "owner"
```

```
}
]
```

Role enum: `owner` | `admin` | `operator` | `viewer`

6. Dashboard

API-DASH-SUMMARY — Overview Metrics

Field	Value
Method / URL	`GET /api/v1/dashboard/summary/`
Auth	JWT
Status	Implemented

Response `200`:

```
{
"cameras_total": 1,
"cameras_online": 1,
"events_today": 12,
"events_pending_review": 5,
"alerts_open": 2,
"alerts_critical": 1,
"edge_devices_online": 1,
"edge_devices_total": 1
}
```

Scoping: Aggregates only resources in organizations where the user has membership.

7. Sites

API-SITE-LIST — List / Create Sites

Field	Value
Method / URL	`GET /api/v1/sites/` / `POST /api/v1/sites/`
Auth	JWT
GET permissions	Member of org
POST permissions	Owner or Admin of target organization
Status	Implemented

Query params (GET):

Param	Description
search	Match `name`, `address`
ordering	`name`, `created_at`

POST request:

```
{
"organization": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
"name": "Warehouse B",
"address": "200 Industrial Rd",
"timezone": "America/New_York",
```

```
"is_active": true
}
```

Validation:

Field	Rules
organization	Required UUID; must exist; user must be Owner/Admin
name	Required; unique per organization
address	Optional string
timezone	Optional; default UTC
is_active	Optional boolean; default true

Response `201`:

```
{
"id": "11111111-2222-3333-4444-555555555555",
"organization": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
"organization_name": "ACME Security Corp",
"name": "Warehouse B",
"address": "200 Industrial Rd",
"timezone": "America/New_York",
"is_active": true,
"created_at": "2026-07-17T07:10:00Z",
"updated_at": "2026-07-17T07:10:00Z"
}
```

Errors: `400` validation; `403` insufficient role; `401` unauthenticated.

API-SITE-DETAIL — Retrieve / Update / Delete Site

Field	Value		
Method / URL	`GET	PATCH	DELETE /api/v1/sites/{id}/`
Auth	JWT		
PATCH/DELETE	Owner or Admin		
Status	Implemented		

PATCH example:

```
{
"address": "200 Industrial Rd, Suite 4",
"is_active": false
}
```

8. Zones

API-ZONE-LIST — List / Create Zones

Field	Value
Method / URL	`GET /api/v1/zones/` / `POST /api/v1/zones/`
Auth	JWT
POST	Owner or Admin of site's organization
Status	Implemented

Query params: `site`, `is_restricted`, `is_active`

POST request:

```
{
"site": "11111111-2222-3333-4444-555555555555",
"name": "Loading Dock",
"polygon_json": [[0.1, 0.2], [0.8, 0.2], [0.8, 0.9], [0.1, 0.9]],
"is_restricted": true,
"is_active": true
}
```

Validation:

Field	Rules
site	Required; must belong to user's org
name	Required; unique per site
polygon_json	Array of `[x,y]` pairs; preferably normalized 0–1
is_restricted	Boolean
is_active	Boolean

Response `201`**:** includes `id`, `site`, `site_name`, `organization`, zone fields, timestamps.

API-ZONE-DETAIL — Retrieve / Update / Delete Zone

Field	Value		
Method / URL	`GET	PATCH	DELETE /api/v1/zones/{id}/`
PATCH/DELETE	Owner or Admin		
Status	Implemented		

9. Cameras

API-CAM-LIST — List / Create Cameras

Field	Value
Method / URL	`GET /api/v1/cameras/` / `POST /api/v1/cameras/`
Auth	JWT
POST	Owner or Admin
Status	Implemented

Query params: `site`, `status`, `is_active`, `search` (name)

POST request:

```
{
"site": "11111111-2222-3333-4444-555555555555",
"zone": "aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee",
"name": "Dock Camera 02",
"rtsp_url": "rtsp://user:pass@192.168.1.50:554/stream1",
"enabled_modules": ["multi_person_tracking", "intrusion_detection"],
"metadata": {"vendor": "Hikvision", "resolution": "1080p"},
"is_active": true
}
```

Validation:

Field	Rules
site	Required UUID
zone	Optional; **should** belong to same site (app validation Planned)
name	Required; unique per site
rtsp_url	Optional string (max 512)
enabled_modules	JSON array of module keys
status / last_seen_at	Read-only on create

Module keys: `face_recognition`, `multi_person_tracking`, `activity_recognition`, `intrusion_detection`, `unknown_person`, `visitor_management`, `attendance`, `fire_smoke`, `weapon_detection`

Response `201`**:**


```
{
"id": "cccccccc-dddd-eeee-ffff-000000000001",
"site": "11111111-2222-3333-4444-555555555555",
"site_name": "Warehouse B",
"organization": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
"zone": "aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee",
"zone_name": "Loading Dock",
"name": "Dock Camera 02",
"rtsp_url": "rtsp://user:pass@192.168.1.50:554/stream1",
"status": "offline",
"enabled_modules": ["multi_person_tracking", "intrusion_detection"],
"last_seen_at": null,
"metadata": {"vendor": "Hikvision", "resolution": "1080p"},
"is_active": true,
"created_at": "2026-07-17T07:20:00Z",
"updated_at": "2026-07-17T07:20:00Z"
}
```


API-CAM-DETAIL — Retrieve / Update / Delete Camera

Field	Value		
Method / URL	`GET	PATCH	DELETE /api/v1/cameras/{id}/`
PATCH/DELETE	Owner or Admin		
Status	Implemented		

10. Edge Devices

API-EDGE-DEV-LIST — List / Register Edge Device

Field	Value
Method / URL	`GET /api/v1/edge-devices/` / `POST /api/v1/edge-devices/`
Auth	JWT
POST	Owner or Admin

Field	Value
Status	Implemented

POST request:

```
{
"site": "11111111-2222-3333-4444-555555555555",
"name": "Edge Node WH-B-01",
"metadata": {"gpu": "RTX 3060"},
"is_active": true
}
```

Response `201` (api_key shown once):

```
{
"id": "edge0001-0000-0000-0000-000000000001",
"site": "11111111-2222-3333-4444-555555555555",
"site_name": "Warehouse B",
"organization": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
"name": "Edge Node WH-B-01",
"status": "offline",
"last_seen_at": null,
"version": "",
"metadata": {"gpu": "RTX 3060"},
"is_active": true,
"api_key_prefix": "AbCdEfGh",
"api_key": "AbCdEfGhIjKlMnOpQrStUvWxYz0123456789_full",
"created_at": "2026-07-17T07:25:00Z",
"updated_at": "2026-07-17T07:25:00Z"
}
```

Security notes:

- Store `api_key` only on the edge node; never log it again.
- DB stores `api_key_hash` (SHA-256), not plaintext.
- Key rotation: Planned.

API-EDGE-HEARTBEAT — Edge Heartbeat

Field	Value
Method / URL	`POST /api/v1/edge/heartbeat/`
Auth	Edge API Key
Status	Implemented

Headers:

```
X-Edge-API-Key: <raw_api_key>
Content-Type: application/json
```

Request:

```
{
"version": "0.1.0",
"status": "online",
"metadata": {"service": "edge-ai", "mode": "simulation"}
}
```

Response `200`:

```
{
"status": "ok",
"registered": true,
"device_id": "edge0001-0000-0000-0000-000000000001"
}
```

If key maps to no device: `{"status":"ok","registered":false}`.

Side effects: Updates `EdgeDevice.last_seen_at`, `status`, `version`; marks site cameras online.

API-EDGE-CAM-LIST — Cameras for Edge Site

Field	Value
Method / URL	`GET /api/v1/edge/cameras/`
Auth	Edge API Key
Status	Implemented

Response `200`:

```
{
"results": [
{
"id": "cccccccc-dddd-eeee-ffff-000000000001",
"name": "Dock Camera 02",
"rtsp_url": "rtsp://...",
"status": "online",
"enabled_modules": ["multi_person_tracking"]
}
]
}
```

Empty `results` if edge key is not bound to a device.

11. Detection Events

API-EVT-LIST — List Events

Field	Value
Method / URL	`GET /api/v1/events/`
Auth	JWT
Status	Implemented

Query params:

Param	Description			
camera	UUID filter			
event_type	Enum filter			
severity	`info` |	`warning` |	`critical`	
review_status	`pending` |	`confirmed` |	`false_positive` |	`escalated`
search	camera name or edge_event_id			
ordering	`created_at`, `confidence`, `severity` (prefix `-` for desc)			
page	Page number			

Response item:

```
{
"id": "evt00001-0000-0000-0000-000000000001",
"camera": "cccccccc-dddd-eeee-ffff-000000000001",
"camera_name": "Dock Camera 02",
```

```
"site_name": "Warehouse B",
"organization": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
"zone": "aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee",
"event_type": "person_detected",
"severity": "warning",
"confidence": 0.87,
"model_name": "yolo-sim",
"model_version": "0.1.0",
"payload": {"track_id": 1, "bbox": [100, 100, 200, 300]},
"review_status": "pending",
"reviewed_by": null,
"reviewed_at": null,
"edge_event_id": "edge-evt-abc-123",
"created_at": "2026-07-17T07:30:00Z",
"updated_at": "2026-07-17T07:30:00Z"
}
```


Event type enum:

`person_detected` | `face_recognized` | `unknown_person` | `intrusion` | `activity` | `fire_smoke` | `weapon` | `attendance` | `visitor`

API-EVT-INGEST — Ingest Event (Edge)

Field	Value
Method / URL	`POST /api/v1/events/ingest/`
Auth	Edge API Key
Status	Implemented

Request:


```
{
"camera_id": "cccccccc-dddd-eeee-ffff-000000000001",
"event_type": "person_detected",
"severity": "warning",
"confidence": 0.87,
"model_name": "yolo-sim",
"model_version": "0.1.0",
"payload": {
"track_id": 1,
"bbox": [100, 100, 200, 300],
"simulated": true
},
"zone_id": "aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee",
"edge_event_id": "edge-evt-abc-123"
}
```


Validation:

Field	Rules
camera_id	Required UUID; active camera
event_type	Required enum
severity	Optional enum; default `info`
confidence	Float 0.0–1.0
model_name	Optional string ≤128
model_version	Optional string ≤64
payload	Optional object
zone_id	Optional UUID
edge_event_id	Optional; used for idempotency

Business rules:

1. Camera must exist and `is_active=true`.
2. If edge user has bound device, camera.site must equal device.site → else `403`.
3. If `edge_event_id` matches existing → return `200` with existing event (idempotent).
4. On create → evaluate alert rules; may create `Alert` rows.
5. Updates camera `last_seen_at` and `status=online`.

Responses:

Status	Meaning
201	New event created
200	Idempotent hit
403	Camera not on edge site
404	Camera not found
400	Validation failure

Remediation Planned: Scope idempotency by organization; reject zone not belonging to camera.site; remove unbound global `EDGE_API_KEY` fallback for ingest.

API-EVT-REVIEW — Review Event

Field	Value	
Method / URL	`GET	PATCH /api/v1/events/{id}/review/`
Auth	JWT	
PATCH	Owner, Admin, or Operator	
Status	Implemented	

PATCH request:

```
{
"review_status": "confirmed"
}
```

Allowed values: `pending` | `confirmed` | `false_positive` | `escalated`

Side effects: Sets `reviewed_by` = current user, `reviewed_at` = now.

12. Alerts

API-ALERT-RULE-LIST — List / Create Alert Rules

Field	Value
Method / URL	`GET /api/v1/alert-rules/` / `POST /api/v1/alert-rules/`
Auth	JWT
POST	Owner or Admin
Status	Implemented

Query params: `event_type`, `is_active`

POST request:

```
{
"organization": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
"name": "Dock Intrusion Critical",
"event_type": "intrusion",
"min_severity": "critical",
"min_confidence": 0.7,
"is_active": true,
"notify_email": true,
"conditions": {"sites": ["11111111-2222-3333-4444-555555555555"]}
}
```

Alert evaluation (Implemented): For each active rule matching `event_type`, if event severity rank ≥ rule min_severity and confidence ≥ min_confidence → create Alert.

Note: `notify_email` and `conditions` are stored but not yet dispatched (notification service Planned).

API-ALERT-LIST — List Alerts

Field	Value
Method / URL	`GET /api/v1/alerts/`
Auth	JWT
Status	Implemented

Query params: `status`, `severity`

Response item:

```
{
"id": "alert001-0000-0000-0000-000000000001",
"organization": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
"event": "evt00001-0000-0000-0000-000000000001",
"event_type": "person_detected",
"camera_name": "Dock Camera 02",
"site_name": "Warehouse B",
"rule": "rule0001-0000-0000-0000-000000000001",
"title": "Person Detected — Dock Camera 02",
"message": "Detection at Warehouse B with confidence 0.87",
"severity": "warning",
"status": "open",
"acknowledged_by": null,
"acknowledged_at": null,
"resolved_at": null,
"created_at": "2026-07-17T07:30:01Z",
"updated_at": "2026-07-17T07:30:01Z"
}
```

API-ALERT-ACK — Acknowledge / Resolve Alert

Field	Value
Method / URL	`PATCH /api/v1/alerts/{id}/acknowledge/`
Auth	JWT
Permissions	Owner, Admin, Operator
Status	Implemented

Request:

```
{
"status": "acknowledged"
}
```

Allowed status values: `open` | `acknowledged` | `resolved` | `dismissed`

Side effects:

- `acknowledged` → set `acknowledged_by`, `acknowledged_at`
- `resolved` or `dismissed` → set `resolved_at`

13. Audit Logs

API-AUDIT-LIST — List Audit Logs

Field	Value
Method / URL	`GET /api/v1/audit-logs/`
Auth	JWT
Status	Partial — endpoint Implemented; auto-write Planned

Query params: `action`, `resource_type`

Response item:

```
{
"id": "aud00001-0000-0000-0000-000000000001",
"organization": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
"actor": 1,
"actor_username": "admin",
"action": "camera.create",
"resource_type": "camera",
"resource_id": "cccccccc-dddd-eeee-ffff-000000000001",
"ip_address": "127.0.0.1",
"details": {},
"created_at": "2026-07-17T07:20:00Z"
}
```

14. Planned Endpoints (M3–M6)

API-ATT-LIST — Attendance Records

`GET /api/v1/attendance/?person_id=&date_from=&date_to=` — JWT, HR+/Admin — Planned

API-VIS-LIST — Visitors

`GET|POST /api/v1/visitors/` — JWT, HR+/Admin — Planned

API-PERSON-ENROLL — Face Enrollment

`POST /api/v1/persons/{id}/enroll/` — JWT, Admin — multipart image + consent_id — Planned

API-RPT-EXPORT — Report Export

`POST /api/v1/reports/export/` — JWT, Admin+ — `{type, date_from, date_to, format}` — Planned

API-EVIDENCE-GET — Evidence Artifact

`GET /api/v1/evidence/{id}/` — JWT, Operator+ — signed URL to object storage — Planned

15. OpenAPI Generation

```
cd backend
python manage.py spectacular --file ../docs/openapi-schema.yml
```

Swagger UI: `http://localhost:8000/api/docs/`

CI requirement: Schema generation must succeed without import errors.

15.1 Example OpenAPI Path Fragment

```
/api/v1/events/ingest/:
post:
operationId: events_ingest_create
tags: [events]
security:
- EdgeApiKey: []
requestBody:
required: true
content:
application/json:
schema:
$ref: '#/components/schemas/EventIngest'
responses:
'201':
description: Created
'200':
description: Idempotent existing event
```

16. Client Integration Notes

Client	Base	Auth
React dashboard	`/api/v1` (proxied)	JWT access in `Authorization`
Edge service	`http://backend:8000/api/v1`	`X-Edge-API-Key`
External integrations	Public HTTPS base	JWT service account Planned

Frontend token storage (current): access token in `localStorage` key `sv_access_token`. Refresh retry Planned.

17. Rate Limits (Planned)

Client	Limit
User JWT	100 req/min
Edge device	1000 req/min
Ingest burst	50 events/sec per edge

18. Remediation Checklist for API Hardening

ID	Issue	Target Fix
API-FIX-01	Global `edge_event_id` idempotency	Unique per organization
API-FIX-02	Unbound EDGE_API_KEY ingest	Require device-bound key
API-FIX-03	Zone/site mismatch on camera/event	Serializer validation

ID	Issue	Target Fix
API-FIX-04	Inconsistent error envelope	Adopt standard error schema
API-FIX-05	Audit not written on mutations	Call `log_audit` in views
API-FIX-06	No rate limiting	Add DRF throttling classes

19. Traceability

API ID	Requirements	Use Cases	Tests
API-AUTH-TOKEN	UC-AUTH-01	—	TEST-API-002
API-CAM-LIST	FR-CAM-01	UC-CAM-01	TEST-API-003
API-EVT-INGEST	FR-ALERT-01	UC-EVT-01	TEST-API-004,005
API-EVT-REVIEW	US-02	UC-EVT-02	Planned
API-ALERT-ACK	FR-ALERT-04	UC-ALERT-01	Planned E2E
API-DASH-SUMMARY	FR-DASH-01	—	TEST-API-006

SentinelVision AI

Volume 06

AI Design Document

Suite Version	1.0.0
Classification	Internal / Confidential
Generated	2026-07-17 07:23 UTC

volume: 06

title: AI Design Document

status: draft

> **SentinelVision AI — Enterprise Documentation Suite**

> Status legend: Implemented | Planned | Future

1. AI Platform Overview

Edge performs stream ingestion and inference; control plane stores metadata, alerts, and evidence references.

Current state: SimulatedYOLODetector produces randomized person_detected events for pipeline validation.

2. Module Catalog

ID	Module	Model	Status	Description
AI-FACE	Face Recognition	InsightFace	Planned	Identify enrolled persons
AI-UNKNOWN	Unknown Person Detection	InsightFace + threshold	Planned	Flag non-enrolled faces
AI-TRACK	Multi-Person Tracking	YOLO + ByteTrack	Planned	Maintain track IDs
AI-INTR	Intrusion Detection	YOLO + zone rules	Planned	Restricted zone entry
AI-ACT	Activity Recognition	YOLO-Pose	Future	Loitering, fall, fight
AI-FIRE	Fire Detection	Custom YOLO	Future	Visual flame cues
AI-SMOKE	Smoke Detection	Custom YOLO	Future	Visual smoke cues
AI-WEAPON	Weapon Detection	Custom YOLO	Future	Firearms/knives
AI-ATT	Attendance	Face + rules	Planned	Clock-in/out
AI-VIS	Visitor Management	Face + registry	Planned	Match pre-registered visitors
AI-VEH	Vehicle Detection	YOLO	Future	Vehicle class detection
AI-LPR	License Plate Recognition	OCR + detector	Future	Plate text extraction

3. Shared Pipeline

```
RTSP -> FrameSampler -> Detector -> Tracker -> ModuleRouter -> EventBuilder -> LocalQueue -> HTTPS
```

4. Module Template

Each module specification includes:

- Input/output contract
- Dataset requirements and licensing
- Model selection criteria
- Confidence thresholds

- Training and evaluation metrics
- Inference flow
- GPU optimization notes
- Versioning and rollback
- Privacy and bias controls

5. AI-FACE — Face Recognition

Attribute	Value
Status	Planned
Model	InsightFace (buffalo_l or production equivalent)
Input	Face crop 112x112+
Output	person_id, confidence, embedding
Threshold	0.75 default; review below 0.85
GPU	Batch inference every N frames
Privacy	Requires BiometricConsent record
Metrics	TAR@FAR 1e-3 on validation set

Inference flow:

1. YOLO person detection
2. Face landmark detection
3. Embedding extraction
4. Nearest neighbor against gallery
5. Emit face_recognized or unknown_person

6. AI-INTR — Intrusion Detection

Attribute	Value
Status	Planned
Input	Track centroid + zone polygon
Rule	Person enters restricted zone during active schedule
Output	intrusion event, severity warning/critical
Threshold	detection confidence >= 0.6

7. AI-WEAPON — Weapon Detection

Attribute	Value
Status	Future
Model	Custom YOLO trained on weapon datasets
Output	weapon event, critical severity

Attribute	Value
Threshold	0.70 minimum before alert
Human review	Mandatory before escalation

8. AI-ATT — Attendance

Attribute	Value
Status	Planned
Input	face_recognized at entry camera
Business rules	One check-in per person per day before cutoff
Output	attendance record linked to person profile

9. Dataset Governance

Requirement	Policy
Licensing	Commercial-use approved datasets only
PII	Anonymize training exports
Bias	Evaluate across age/gender/skin tone buckets
Retention	Training snapshots deleted after model promotion

10. Model Versioning

Field	Storage
model_name	DetectionEvent.model_name
model_version	DetectionEvent.model_version
registry	Planned ai_modelregistry table
rollback	Edge OTA package pin to previous version

11. GPU Optimization

- FP16 inference where supported
- Frame skipping adaptive to GPU load
- Batch face embedding extraction
- Limit concurrent RTSP decodes to hardware capacity

12. Performance Metrics

Module	Precision Target	Recall Target	FPS Target
Person detection	0.90	0.88	15+ @1080p
Face recognition	0.95	0.90	10+
Intrusion	0.92	0.90	N/A rule-based
Weapon	0.85	0.80	10+

13. Current Simulation

SimulatedYOLODetector emits person_detected with random confidence and bbox for integration testing only. Not for production security decisions.

SentinelVision AI

Volume 07

UI/UX Specification

Suite Version	1.0.0
Classification	Internal / Confidential
Generated	2026-07-17 07:23 UTC

volume: 07

title: UI/UX Specification

status: draft

> **SentinelVision AI — Enterprise Documentation Suite**

> Status legend: Implemented | Planned | Future

1. Design Principles

- Security-first dark operations theme
- Responsive layout for desktop and mobile browsers
- Clear severity signaling for alerts and events
- Minimize alert fatigue via prioritization and filters
- WCAG 2.1 AA compliance target

2. Information Architecture

```
Login -> App Shell
Overview
Cameras
Events
Alerts
Reports (Planned)
Attendance (Planned)
Visitors (Planned)
Users (Planned)
Settings (Planned)
```

3. Screen Catalog

ID	Screen	Status	Description
UI-LOGIN	Login	Implemented	JWT authentication entry
UI-DASH	Dashboard Overview	Implemented	Summary metric cards
UI-CAM	Camera Management	Partial	Read-only inventory list
UI-LIVE	Live View	Planned	Low-latency stream grid
UI-EVT	Event Review	Partial	Review queue with confirm/false+
UI-ALERT	Alerts	Partial	Alert cards with acknowledge
UI-RPT	Reports	Planned	Export and schedule reports
UI-ATT	Attendance	Planned	Daily attendance dashboard
UI-VIS	Visitor Management	Planned	Pre-register and check-in
UI-USR	User Management	Planned	Membership and role admin
UI-SET	Settings	Planned	Org settings, retention, modules
UI-EDGE	Edge Health	Planned	Device status and diagnostics

4. UI-LOGIN

Purpose: Authenticate operator and establish JWT session.

Components: username field, password field, submit button, error banner, demo hint.

States: idle, loading, error, success redirect.

Wireframe (ASCII):

```
+----------------------------+
| SentinelVision AI |
| [ username ] |
| [ password ] |
| [ Sign in ] |
+----------------------------+
```

5. UI-DASH

Metrics: cameras online/total, events today, pending review, open/critical alerts, edge online/total.

Interaction: read-only cards; auto-refresh Planned.

6. UI-EVT

Table columns: time, type, camera, site, severity, confidence, status, actions.

Actions: Confirm, False Positive (Operator+).

Planned: filters, pagination controls, evidence thumbnail preview.

7. Component Library

Component	Usage
MetricCard	Dashboard KPIs
DataTable	Cameras, events
AlertCard	Alerts list
Badge	status/severity
SidebarNav	Primary navigation
ErrorBanner	API failures

8. Navigation Flow

Login success -> Overview -> Events/Alerts for investigation -> Acknowledge alert -> Optional review confirmation.

9. Responsive Behavior

- Desktop: persistent sidebar
- Mobile: stacked sidebar/nav wrap; tables horizontally scrollable

10. Accessibility

- Keyboard navigable form controls
- Color contrast >= 4.5:1 for text
- Planned: focus indicators, ARIA labels on tables/buttons

11. PWA

Manifest present; service worker and push notifications Planned.

SentinelVision AI

Volume 08

DevOps and Deployment

Suite Version	1.0.0
Classification	Internal / Confidential
Generated	2026-07-17 07:23 UTC

volume: 08

title: DevOps and Deployment

status: draft

> **SentinelVision AI — Enterprise Documentation Suite**

> Status legend: Implemented | Planned | Future

1. Environments

Environment	Purpose	Status
Local	Developer workstation	Implemented
CI	Automated validation	Implemented
Staging	Pre-production	Planned
Production	Customer-facing	Planned

2. CI/CD — GitHub Actions

Current pipeline jobs:

- backend: migrate + pytest
- edge: pytest
- frontend: lint + build
- prd/docs: documentation build

Planned gates: coverage thresholds, security scan, container scan, staging deploy.

3. Docker

Image	Base	Purpose
backend	python:3.12-slim	Django API
frontend	node:20-alpine + nginx	Dashboard
edge	python:3.12-slim	Edge worker

4. Docker Compose

Services: db, backend, frontend, edge with healthchecks and persistent postgres volume.

5. Kubernetes (Planned)

- Helm chart with values per environment
- Separate namespaces for staging/production
- Secrets via external secrets operator

6. Nginx

- Serve frontend static assets
- Proxy `/api/` to backend
- Planned: TLS termination, rate limits, WAF integration

7. SSL/TLS

- Local dev: HTTP
- Production: TLS 1.2+ with auto-renewed certificates

8. Redis & Celery (Planned)

Worker deployment for notifications, reports, retention cleanup.

9. Monitoring

Component	Tool	Status
Metrics	Prometheus	Planned
Dashboards	Grafana	Planned
Logs	ELK	Planned
Uptime	Health checks	Partial

10. Backup & DR

Asset	RPO	RTO	Method
PostgreSQL	15 min	1 hr	Managed backups + PITR
Object storage	1 hr	2 hr	Cross-region replication
Config/secrets	24 hr	4 hr	Git + vault restore

11. Security Hardening

- Non-root containers
- Secret rotation policy
- Network policies in Kubernetes
- Dependabot and bandit in CI (Planned)

12. Runbooks

Scenario	Action
API unhealthy	Check DB connectivity, restart pods

Scenario	Action
Edge offline	Verify outbound HTTPS, API key, RTSP source
Ingest backlog	Scale API replicas, inspect edge queue
DB failover	Promote replica, update connection string

SentinelVision AI

Volume 09

Testing Documentation

Suite Version	1.0.0
Classification	Internal / Confidential
Generated	2026-07-17 07:23 UTC

volume: 09

title: Testing Documentation

status: draft

> **SentinelVision AI — Enterprise Documentation Suite**

> Status legend: Implemented | Planned | Future

1. Test Strategy

Follow the test pyramid: unit -> integration/API -> E2E -> performance/security/UAT.

2. Current Baseline

Suite	Count	Status
Backend API tests	5	Implemented
Edge tests	2	Implemented
Frontend tests	1	Implemented

Coverage target for core backend: >= 80% (Planned enforcement).

3. Test Environments

Env	Data	Usage
Local SQLite/Postgres	seed_dev_data	Developer
CI ephemeral	migrations + factories	PR validation
Staging	anonymized	Integration/E2E

4. Test Case Catalog

ID	Test	Status	Requirement Ref
TEST-API-001	Health endpoint returns healthy	Implemented	FR-DASH-01
TEST-API-002	JWT login returns tokens	Implemented	UC-AUTH-01
TEST-API-003	Tenant camera isolation	Implemented	BR-TEN-01
TEST-API-004	Event ingest creates event	Implemented	UC-EVT-01
TEST-API-005	Duplicate edge_event_id idempotent	Implemented	UC-EVT-01
TEST-API-006	Dashboard summary metrics	Implemented	FR-DASH-01
TEST-EDGE-001	Simulated detector structure	Implemented	AI-TRACK
TEST-EDGE-002	Event payload builder	Implemented	API-EVT-INGEST

ID	Test	Status	Requirement Ref
TEST-UI-001	API base defaults to /api/v1	Implemented	UI-DASH
TEST-API-007	RBAC denies viewer config write	Planned	FR-CAM-01
TEST-API-008	Cross-tenant ingest forbidden	Planned	BR-EDGE-01
TEST-E2E-001	Login to alert acknowledgement	Planned	UC-ALERT-01
TEST-AI-001	YOLO precision on validation set	Planned	AI-FACE
TEST-PERF-001	Ingest 100 events/sec soak	Planned	NFR-01
TEST-SEC-001	OWASP API top 10 scan	Planned	NFR-05

5. Unit Testing

- Django models, serializers, permissions, alert rule evaluation
- Edge inference helpers and payload builder

6. Integration/API Testing

- DRF APIClient with JWT and edge headers
- Tenant isolation across all list endpoints (Planned expansion)

7. E2E Testing (Planned)

Playwright flows: login, dashboard load, event review, alert acknowledgement.

8. AI Validation (Planned)

- Fixed benchmark clips per module
- Precision/recall/F1 reporting
- False positive rate monitoring in staging

9. Performance Testing (Planned)

Locust scenarios for event ingest and dashboard read load.

10. Security Testing (Planned)

- Bandit static analysis
- Auth bypass attempts
- Tenant isolation fuzz tests
- Pen test before production

11. UAT

Entry criteria: staging deploy green, P0 test cases pass, known defects triaged.

Exit criteria: stakeholder sign-off on intrusion/alert workflows.

12. Defect Severity

Level	Definition
S1	Production down or security breach
S2	Major feature broken, no workaround
S3	Minor feature issue with workaround
S4	Cosmetic/documentation

SentinelVision AI

Volume 10

Project Management

Suite Version	1.0.0
Classification	Internal / Confidential
Generated	2026-07-17 07:23 UTC

volume: 10

title: Project Management

status: draft

> **SentinelVision AI — Enterprise Documentation Suite**

> Status legend: Implemented | Planned | Future

1. Program Overview

Deliver SentinelVision AI from MVP foundation to enterprise-ready hybrid security platform in phased releases.

2. Team Structure

Role	Responsibility
Product Manager	Priorities, acceptance, stakeholder communication
Tech Lead	Architecture, code review, technical risk
Backend Engineers	Django API, data model, integrations
Frontend Engineers	Dashboard, UX implementation
ML/CV Engineers	Edge inference modules
DevOps Engineer	CI/CD, infrastructure, observability
QA Engineer	Test planning and automation
Security/Compliance	Privacy, audits, hardening

3. Epics

Epic	Description	Phase
E1 Foundation	Auth, tenancy, cameras, docs, CI	M0/M1
E2 Event Pipeline	RTSP, YOLO, ingest hardening	M2
E3 Biometrics	Face, attendance, visitors	M3
E4 Threat Detection	Weapon/fire/smoke	M4
E5 Enterprise Ops	Reports, notifications, HA, SSO	M5

4. Sprint Plan (Indicative)

Sprint	Goals	Points
S1	Documentation suite + tenant hardening	34
S2	RTSP adapter + real YOLO integration	40
S3	Zone intrusion + alert UX	36

Sprint	Goals	Points
S4	Face enrollment + attendance	42
S5	Notifications + reports	38
S6	Production K8s + observability	45

Story point scale: Fibonacci, team velocity assumed 35-40 points/sprint.

5. Timeline

Milestone	Target	Exit Criteria
M0/M1 Complete	Done	Foundation deployed locally
M2 Core AI	Week 10	RTSP + person detection + intrusion
M3 Biometrics	Week 14	Attendance + visitor check-in
M4 Threat	Week 18	Weapon/fire/smoke modules
M5 Enterprise	Week 24	HA, SSO, reporting, DR tested

6. Budget Assumptions

Category	Estimate (USD)	Notes
Engineering (6 FTE x 6 mo)	$450k-$600k	Region dependent
GPU edge hardware pilot	$15k-$40k	3-5 sites
Cloud infrastructure	$3k-$8k/mo	Staging + prod
Third-party services	$1k-$3k/mo	SMS, email, monitoring
Security audit	$25k-$50k	Pre-production

7. Risks

ID	Risk	Impact	Mitigation
RISK-01	False positive alert fatigue	High	Threshold tuning, human review
RISK-02	Privacy non-compliance	High	Consent workflows, legal review
RISK-03	Edge hardware variability	Medium	Certified hardware list
RISK-04	Stream instability	High	Reconnect + health alerts
RISK-05	Model bias	Medium	Fairness evaluation
RISK-06	Tenant isolation defect	Critical	Expand isolation tests, remediate known gaps

8. Release Plan

Release	Scope
v0.1.0	MVP foundation (current)
v0.2.0	RTSP + YOLO + intrusion
v0.3.0	Face + attendance + visitors
v0.4.0	Threat modules + reports
v1.0.0	Enterprise HA, SSO, production SLOs

9. Governance

- Weekly sprint review and demo
- Bi-weekly architecture review
- Change control board for schema and API breaking changes
- Documentation updated per release tag

10. Final Deliverable Acceptance

- All P0 requirements Implemented or formally deferred
- Documentation suite regenerated and manifest published
- CI green on main branch
- Stakeholder UAT sign-off
- Production runbooks and DR test evidence